Automated Root Cause Analysis (RCA) System

for Production Failures

A Comprehensive AI-Powered Solution

Project Classification

Final Year Computer Science/Engineering Project
Domain: AI/ML, Systems Engineering, DevOps, SRE

Technical Complexity: Advanced
Implementation Duration: 16-24 weeks

Document Version 1.0

February 2026

Table of Contents

Table of Contents 3
Executive Summary 5
1. Introduction and Motivation 6
1.1 The Production Failure Challenge 6
1.2 Limitations of Traditional Approaches 6
1.3 The Automated RCA Solution 7
2. Core Concept: How the System Works 8
2.1 Medical Diagnostician Analogy 8
2.2 Four-Stage Processing Pipeline 8
Stage 1: Normal Behavior Learning 8
Stage 2: Anomaly Detection 8
Stage 3: Causal Inference 9
Stage 4: Root Cause Ranking 9
3. Feasibility Analysis 10
3.1 Technical Feasibility: 85% (Highly Feasible) 10
Strengths Supporting Feasibility 10
Challenges and Mitigation Strategies 10
3.2 Project Scope Appropriateness 10
4. Comprehensive Anomaly Detection Catalog 12
4.1 CPU & Compute Anomalies (10 types) 12
4.2 Memory Anomalies (10 types) 12
4.3 Storage & I/O Anomalies (10 types) 13
4.4 Network Anomalies (10 types) 13
4.5 Application Layer Anomalies (10 types) 14
5. Case Studies: AI-Powered Root Cause Analysis in Action 15
Case Study 1: E-Commerce Platform - Database Migration Induced Failure 15
5.1.1 Scenario Background 15
5.1.2 Timeline of Events 15
5.1.3 AI System Analysis Process 15
5.1.4 AI-Generated Root Cause Report 16
Case Study 2: SaaS Platform - Memory Leak Causing Cascading Failure 18
5.2.1 Scenario Background 18
5.2.2 AI Detection: Memory Leak Pattern Recognition 18

5.2.3 Causal Analysis: Tracing to Code Deployment 18
5.2.4 Root Cause Report & Remediation 18
Case Study 3: Financial Services - Network Partition Causing Split-Brain 19
5.3.1 Scenario Background 19
5.3.2 AI Detection: Multi-Metric Correlation 19
5.3.3 Causal Graph Analysis 19
5.3.4 Root Cause & Resolution 19
Case Study 4: IoT Platform - Thread Pool Exhaustion in Manufacturing System 21
5.4.1 Scenario Background 21
5.4.2 AI Analysis: Rapid Root Cause Identification 21
5.4.3 Causal Inference Results 21
5.4.4 AI-Generated Report 21
Case Study 5: Cloud Storage - DNS Propagation Delay Causing Regional Outage . 22
5.5.1 Scenario Background 22
5.5.2 AI Detection: Partial Failure Pattern 22
5.5.3 Causal Analysis: Event Correlation 22
5.5.4 AI-Generated Diagnosis 22
6. Technical Implementation Details 23
6.1 Technology Stack 23
6.2 System Architecture Overview 23
7. Expected Outcomes & Success Criteria 24
7.1 Performance Metrics & Targets 24
7.2 Three Levels of Project Success 24
Pass Level - Minimum Viable Product 24
Distinction Level - Production-Quality System 24
Publication-Worthy - Research Contribution 24
8. Conclusion 26
8.1 Why This Project Stands Out 26
8.2 Key Takeaways from Case Studies 26
8.3 Next Steps for Implementation 26
8.4 Final Thoughts 27

Executive Summary

In modern distributed computing environments, production failures rarely occur due to a single, isolated event. Instead, they emerge from complex cascading interactions between system components, configuration changes, resource constraints, and temporal delays. Traditional monitoring systems excel at detecting that something has failed but fall short in explaining why the failure occurred and what triggered it.

This document presents a comprehensive design and implementation guide for an AI-powered Automated Root Cause Analysis (RCA) system that fundamentally transforms how organizations diagnose production failures. By integrating deep learning-based temporal models (LSTM/Transformers), unsupervised anomaly detection, and causal inference techniques, the system automatically traces failure chains back to their root causes, providing actionable explanations rather than simple alerts.

Key Capabilities:

- **Intelligent Learning:** Learns normal operational behavior from healthy system data without requiring labeled failure examples
- **Advanced Detection:** Identifies 50+ distinct anomaly types across infrastructure, network, and application layers
- **Causal Analysis:** Distinguishes correlation from causation using Granger causality and causal graph construction
- **Actionable Insights:** Provides ranked root causes with confidence scores and detailed causal chains
- **Real-time Processing:** Delivers analysis within minutes of failure occurrence

Core Innovation: The system moves beyond reactive symptom detection to proactive root cause explanation through causal temporal modeling, enabling organizations to understand not just what failed, but why it failed and how the failure propagated through their infrastructure.

Target Applications: Cloud platforms (AWS, Azure, GCP), microservices architectures, data pipelines, e-commerce systems, financial services, and manufacturing IoT environments.

1. Introduction and Motivation

1.1 The Production Failure Challenge

Modern distributed systems operate at unprecedented scale and complexity. A typical e-commerce platform might consist of hundreds of microservices, thousands of containers, and millions of transactions per day. When failures occur in such environments, engineers face a daunting diagnostic challenge:

- **Metric Overload:** During a typical production incident, 50-100+ metrics spike simultaneously. Distinguishing symptoms from causes becomes nearly impossible through manual analysis.
- **Cascading Failures:** A single root cause triggers a domino effect across multiple services. For example, a database slow query → connection pool exhaustion → API timeouts → retry storms → memory exhaustion → complete service outage.
- **Temporal Complexity:** Root causes and visible symptoms are often separated by significant time delays. A configuration change deployed at 2:00 PM might only manifest as a user-facing failure at 8:00 PM during peak traffic.
- **Investigation Time:** Engineers typically spend 2-8 hours manually correlating logs, metrics, deployment events, and configuration changes to identify root causes. During this time, business impact continues to accumulate.
- **Knowledge Dependency:** Effective diagnosis requires deep institutional knowledge of system architecture, historical failure patterns, and component interactions. This expertise is scarce and not always available during critical incidents.

1.2 Limitations of Traditional Approaches

Approach	Method	Critical Limitation
Static Thresholds	Alert when CPU > 80% or memory > 90%	Cannot explain WHY threshold exceeded; high false positive rate
Rule-Based Systems	IF metric_X AND event_Y THEN root_cause_Z	Brittle; cannot adapt to new failure modes; exponential rule explosion
Correlation Analysis	Find metrics correlated with failure	Cannot distinguish cause from effect; flags 100+ correlated metrics
Manual Investigation	Human experts analyze dashboards	Slow (hours/days); inconsistent; not scalable; expertise-dependent

1.3 The Automated RCA Solution

This project proposes an intelligent system that addresses these limitations through a multi-stage AI pipeline:

1. **Behavioral Learning Phase:** Continuously observe system metrics during normal operation and build deep learning models (LSTM/Transformers) that understand typical patterns and relationships.
2. **Anomaly Detection Phase:** When failures occur, identify which specific metrics deviate from learned normal behavior using reconstruction error and forecast deviation techniques.
3. **Causal Inference Phase:** Apply Granger causality tests and causal graph construction to determine which anomalous events caused which downstream effects, distinguishing symptoms from root causes.
4. **Root Cause Ranking Phase:** Score and rank candidate root causes based on causal strength, temporal precedence, and anomaly severity.
5. **Explanation Generation Phase:** Produce human-readable causal chains, confidence scores, and supporting evidence that engineers can use for remediation.

2. Core Concept: How the System Works

2.1 Medical Diagnostician Analogy

The RCA system operates analogously to how a physician diagnoses disease:

Medical Diagnosis	RCA System
Learn what “healthy” vital signs look like through training	Learn normal system behavior from operational metrics
Detect symptoms (fever, elevated heart rate)	Detect anomalous metrics (high latency, CPU spikes)
Trace symptoms to underlying disease (bacterial infection)	Trace anomalies to root cause (database migration)
Prescribe treatment based on diagnosis	Recommend remediation (rollback migration, add index)

2.2 Four-Stage Processing Pipeline

Stage 1: Normal Behavior Learning

The system continuously ingests time-series metrics (sampled every 1-5 minutes) from production systems:

- Infrastructure: CPU utilization, memory usage, disk I/O, network throughput
- Application: API latency (P50/P95/P99), error rates, request throughput, queue depths
- Database: Connection pool usage, query latency, transaction rates, lock wait time
- Cache: Hit/miss rates, eviction rates, memory consumption

An LSTM-based autoencoder neural network trains on 30-90 days of healthy operational data, learning to predict normal metric patterns. The model captures complex relationships such as:

- “When API request rate increases 2x, CPU typically rises 40% and database connections increase by 30%”
- “Code deployments cause a brief 200ms latency spike for 5 minutes, then return to baseline”
- “During business hours (9 AM - 5 PM), database connections peak at 85/100; overnight they drop to 20/100”

Stage 2: Anomaly Detection

When a failure occurs, the system uses the trained model to identify anomalous deviations:

Reconstruction Error Method: The model attempts to reconstruct the current metric sequence. Large reconstruction errors indicate the pattern doesn’t match learned normal behavior.
Forecast Deviation Method: The model predicts the next N time steps. Significant deviations between predicted and actual values signal anomalies.
Example: During a production incident, the system might identify:

- **API latency:** Expected 150ms, observed 5000ms → Anomaly score: 0.96
- **DB connections:** Expected 65/100, observed 100/100 → Anomaly score: 0.89
- **Error rate:** Expected 0.01%, observed 15% → Anomaly score: 0.98
- **CPU usage:** Expected 45%, observed 50% → Anomaly score: 0.12 (not anomalous)

Stage 3: Causal Inference

The system distinguishes correlation from causation using statistical causal inference:
Granger Causality Tests: For each pair of metrics, test whether past values of metric A improve prediction of metric B. If yes, A Granger-causes B.
Temporal Precedence Analysis: Causes must occur before effects. The system identifies which anomalies appeared first in the timeline.
Causal Graph Construction: Build a directed graph where nodes are metrics/events and edges represent causal relationships. Use PC algorithm to learn graph structure from data.
Example causal chain discovered:
Database Query Latency (18:30) → Connection Pool Exhaustion (20:45) → API Timeout (21:00) → User Error Rate (21:00)

Stage 4: Root Cause Ranking

The system scores each anomalous metric/event as a potential root cause using:

- **Causal Outflow:** How many downstream effects does this cause? (High = likely root)
- **Causal Inflow:** How many upstream causes does this have? (Low = likely root)
- **Temporal Priority:** How early did this anomaly appear? (Earlier = higher priority)
- **Event Correlation:** Did a deployment or config change occur near this anomaly?

Final output: Ranked list with confidence scores and explanations.

3. Feasibility Analysis

3.1 Technical Feasibility: 85% (Highly Feasible)

Strengths Supporting Feasibility

- **Mature Technology Stack:** PyTorch/TensorFlow for deep learning are production-ready with extensive documentation and community support.
- **Academic Validation:** Techniques validated in published research from Google (Vertex), Microsoft (FluxInfer), and academic institutions.
- **Data Availability:** Cloud platforms (Prometheus, CloudWatch, Datadog) provide extensive metrics APIs. Synthetic data can be generated for development.
- **Computational Resources:** Free GPU resources available via Google Colab for model training. Production deployment requires modest resources (4GB RAM, 2 CPU cores).
- **Incremental Development:** Can start with MVP (5-10 metrics, synthetic data) and progressively add complexity.

Challenges and Mitigation Strategies

Challenge	Mitigation Strategy
Data quality varies in real systems (missing values, irregular sampling)	Implement robust preprocessing: forward-fill short gaps, interpolate medium gaps, flag long gaps for manual review
Causal inference computationally expensive at scale ($O(n^2)$ for n metrics)	Limit analysis to top-N anomalous metrics; use parallel processing; optimize with Cython/C++ for bottlenecks
Requires domain knowledge for dependency graphs and metric selection	Provide configuration templates for common architectures; auto-discover dependencies from service mesh metadata; allow incremental refinement
Model interpretability challenging (deep learning black box)	Use attention mechanisms to visualize important features; provide reconstruction error per metric; validate with synthetic data ground truth
Validation difficult without ground truth in production	Use public benchmarks (NASA, Azure datasets); generate synthetic failures with known causes; compare to post-mortem reports

3.2 Project Scope Appropriateness

This project is exceptionally well-suited for a final-year submission:

- **Multi-Disciplinary Integration:** Combines machine learning, statistics (causal inference), systems engineering, and software development.
- **Real-World Relevance:** Addresses actual pain points at Google, Netflix, Amazon, and every company running production systems.
- **Measurable Outcomes:** Clear success metrics (precision, recall, top-k accuracy) enable objective evaluation.
- **Scalable Complexity:** Minimum viable product achievable in 12 weeks; can be extended to 24+ weeks for publication-quality work.
- **Career Impact:** Directly demonstrates skills needed for Site Reliability Engineer, DevOps Engineer, and ML Engineer roles. Excellent portfolio piece for job interviews.
- **Publication Potential:** Novel contributions in causal inference or architecture could lead to workshop/conference papers.

4. Comprehensive Anomaly Detection Catalog

The RCA system is designed to detect and analyze 50+ distinct anomaly types across five major categories. Each anomaly has unique detection signatures and typical root causes.

4.1 CPU & Compute Anomalies (10 types)

#	Anomaly Type	Detection Signature	Typical Root Cause
1	**CPU Saturation**	Sustained CPU >90% beyond normal traffic patterns	Runaway process, infinite loop, compute-heavy task
2	**CPU Oscillation**	Rapid fluctuation between high and low CPU usage	Misconfigured auto-scaling, task thrashing
3	**Single-Core Bottleneck**	One core at 100%, others idle	Single-threaded application on multi-core system
4	**CPU Steal Time Spike**	High steal time in virtualized environment	Cloud neighbor resource contention (noisy neighbor)
5	**Context Switch Storm**	Excessive OS context switches	Too many threads, poor thread scheduling
6	**CPU Affinity Misconfig**	Process pinned to wrong cores	Incorrect CPU binding configuration
7	**Thermal Throttling**	CPU frequency reduction due to heat	Cooling system failure, data center AC issue
8	**Nice Value Anomaly**	Critical process starved by low-priority tasks	Process priority misconfiguration
9	**Interrupt Storm**	Excessive hardware interrupts	Faulty network card, disk controller malfunction
10	**CPU Cache Miss Rate**	L2/L3 cache misses increase significantly	Poor memory access pattern, code change affecting locality

4.2 Memory Anomalies (10 types)

Memory-related anomalies often manifest gradually and can cause cascading failures:

- **11. Memory Leak:** Gradual monotonic increase without release → Application bug, unclosed resources

- **12. Out of Memory (OOM):** System kills processes → Memory leak, traffic surge, undersized instance
- **13. Swap Thrashing:** Excessive paging to disk → Insufficient RAM, memory leak
- **14. Page Fault Storm:** High rate of page faults → Working set exceeds RAM
- **15. Memory Fragmentation:** Available memory but allocation fails → Long-running process, frequent alloc/free
- **16. Shared Memory Leak:** /dev/shm exhausted → Orphaned shared memory segments
- **17. Buffer/Cache Explosion:** Kernel caches consuming excessive memory → Filesystem operations without pressure
- **18. Huge Pages Misconfig:** THP causing periodic latency → Transparent Huge Pages defragmentation
- **19. Memory Bandwidth Saturation:** Memory bus saturated → Excessive concurrent memory access
- **20. NUMA Node Imbalance:** Uneven memory distribution → Poor NUMA affinity settings

4.3 Storage & I/O Anomalies (10 types)

- **21. Disk Space Exhaustion:** Storage >95% full → Log accumulation, no rotation
- **22. I/O Wait Spike:** Processes blocked waiting for disk → Slow disk, RAID rebuild
- **23. Disk Latency Increase:** Read/write latency >10x baseline → Disk degradation, controller issue
- **24. IOPS Saturation:** Disk operations maxed → Excessive small random reads/writes
- **25. I/O Pattern Shift:** Sequential to random pattern change → Query plan change, fragmentation
- **26. Inode Exhaustion:** No free inodes despite disk space → Too many small files
- **27. Write Amplification:** SSD writes >> app writes → Poor write patterns, SSD aging
- **28. Read-Ahead Misconfig:** Inefficient prefetching → Incorrect blockdev settings
- **29. RAID Degradation:** Array running degraded → Disk failure in array
- **30. Filesystem Corruption:** Journal errors → Improper shutdown, hardware fault

4.4 Network Anomalies (10 types)

- **31. Packet Loss:** Packets dropped >1% → Network congestion, faulty switch
- **32. Bandwidth Saturation:** Network interface at capacity → Traffic surge, DDoS
- **33. TCP Retransmissions:** High retransmission rate → Network instability, congestion
- **34. Connection Reset Storm:** TCP connections abruptly closed → Load balancer issue, firewall

- **35. DNS Resolution Failure:** DNS lookups slow/failing → DNS server issue, network partition
- **36. NAT Table Exhaustion:** NAT table full → Too many concurrent connections
- **37. MTU Mismatch:** Packet fragmentation → Path MTU discovery failure
- **38. ARP Cache Overflow:** ARP table exceeds capacity → Too many hosts in subnet
- **39. Interface Errors:** CRC errors, frame errors → Faulty cable, NIC hardware
- **40. Asymmetric Routing:** Packets take different paths → Routing misconfiguration

4.5 Application Layer Anomalies (10 types)

- **41. Thread Pool Exhaustion:** All worker threads busy → Slow downstream, blocking operations
- **42. Connection Pool Saturation:** DB connections maxed → Connection leak, insufficient pool
- **43. Garbage Collection Pause:** Long GC pauses → Heap size issue, GC tuning needed
- **44. Event Loop Blocking:** Node.js/async loop blocked → Synchronous operation in async code
- **45. Circuit Breaker Activation:** Dependency protection triggered → Downstream service failure
- **46. Rate Limiter Throttling:** Requests rejected → Traffic surge, DDoS, crawler
- **47. Cache Stampede:** Simultaneous cache misses → Popular cache entry expired
- **48. Deadlock:** Threads waiting indefinitely → Lock acquisition order bug
- **49. Semaphore Contention:** Concurrent shared resource access → Resource bottleneck
- **50. Message Queue Backlog:** Messages accumulating → Consumer slow, producer fast

5. Case Studies: AI-Powered Root Cause Analysis in Action

The following five case studies demonstrate how the RCA system detects anomalies, performs causal inference, and provides actionable root cause explanations across different failure scenarios.

Case Study 1: E-Commerce Platform - Database Migration Induced Failure

5.1.1 Scenario Background

System: Large e-commerce platform handling 10,000 requests/minute during peak hours
Architecture: Microservices (Frontend API, Product Service, Order Service, PostgreSQL Database, Redis Cache)
Incident: Complete service outage affecting customer checkout functionality
Business Impact: $50,000 revenue loss over 45-minute outage

5.1.2 Timeline of Events

Time	Event Description
14:00	**Database schema migration deployed:** Added composite index on product_variants table (50M rows)
14:00-18:00	System appears normal. Low traffic period, migration impact not visible.
18:30	**First anomaly detected:** Product query latency increases from 50ms → 200ms. PostgreSQL query planner chose inefficient index after migration.
19:30	**Second anomaly:** Product query latency reaches 400ms. Traffic increasing as evening shoppers arrive.
20:45	**Cascade begins:** Database connection pool reaches 95/100. Query latency now 500ms. Slow queries consuming connections.
21:00	**Complete failure:** Connection pool saturated (100/100). API requests timing out. Error rate spikes to 15%. Customer-facing outage begins.

5.1.3 AI System Analysis Process

<u>Step 1: Anomaly Detection</u>

The LSTM model, trained on 60 days of healthy data, flagged the following anomalies at 21:05 (5 minutes after failure onset):

Metric	Expected	Actual	Score
api_latency_p95	180ms	**5000ms**	**0.96**
db_connections_active	65/100	**100/100**	**0.89**
product_query_latency	60ms	**500ms**	**0.91**
error_rate	0.01%	**15.0%**	**0.98**

Step 2: Causal Inference

Granger causality tests revealed temporal causal relationships:

- **product_query_latency** → **db_connections_active** ($p = 0.002$, lag = 2 hours)
- **db_connections_active** → **api_latency_p95** ($p = 0.001$, lag = 15 minutes)
- **api_latency_p95** → **error_rate** ($p < 0.0001$, lag = 0 minutes, simultaneous)

Event correlation identified: db_migration_applied event at 14:00, 4.5 hours before product_query_latency anomaly.

Step 3: Root Cause Ranking

The system scored candidates:

Root Cause Candidate	Confidence	Causal Out	Time Priority
Database Schema Migration	**87%**	High (3)	7 hrs before
Connection Pool Size	23%	Low (1)	No change
Traffic Volume Increase	12%	None (0)	Normal range

5.1.4 AI-Generated Root Cause Report

═══ ROOT CAUSE ANALYSIS REPORT ═══

Generated: 21:05 (5 minutes after failure onset)

PRIMARY ROOT CAUSE (87% confidence):

Database schema migration deployed at 14:00

CAUSAL CHAIN:

1. 14:00 - Composite index added to product_variants table
2. 18:30 - PostgreSQL query planner chose new (inefficient) index
3. 18:30 - Product queries slowed from 50ms → 500ms (10x increase)
4. 20:45 - Slow queries consumed database connections (pool exhaustion)
5. 21:00 - API requests timed out waiting for database connections
6. 21:00 - User-facing error rate spiked to 15% (service outage)

SUPPORTING EVIDENCE:

- Query execution time increased 10x (50ms → 500ms) after migration
- Granger causality: product_query_latency → db_connections (p=0.002)
- Temporal precedence: Migration 7 hours before visible failure
- No other configuration changes in 24-hour window
- Traffic volume within normal range (not a capacity issue)

IMMEDIATE REMEDIATION:

- **PRIORITY 1:** Rollback database schema migration (5 min)
- **PRIORITY 2:** Force PostgreSQL to use original index with query hint (immediate)
- **SHORT-TERM:** Increase connection pool size to 200 (mitigation)

LONG-TERM PREVENTION:

- Require query performance benchmarks before schema migrations
- Deploy database changes during low-traffic windows only
- Implement canary deployment for schema changes (5% traffic first)
- Add automated query performance regression tests to CI/CD

Case Study 2: SaaS Platform - Memory Leak Causing Cascading Failure

5.2.1 Scenario Background

System: Multi-tenant SaaS analytics platform processing 1M events/hour
Architecture: Node.js API servers (20 instances), Kafka message queue, ElasticSearch, Redis
Incident: Gradual degradation over 48 hours culminating in complete service failure
Detection Challenge: Slow-onset failure; symptoms appeared long after root cause introduced

5.2.2 AI Detection: Memory Leak Pattern Recognition

The LSTM model identified an unusual memory growth pattern:

- **Normal behavior:** Memory oscillates between 1.2GB-1.8GB with periodic GC drops
- **Anomaly detected:** Monotonic upward trend from 1.5GB → 3.8GB over 36 hours
- **Anomaly score:** 0.94 (very high confidence)
- **Pattern type:** Trend anomaly (vs. spike anomaly)

5.2.3 Causal Analysis: Tracing to Code Deployment

The system performed temporal event correlation:

- **Event:** Code deployment Monday 10:00 AM (v2.14.3)
- **Memory anomaly start:** Monday 14:00 PM (4 hours post-deployment)
- **Correlation strength:** 0.89 (high temporal correlation)
- **Code change isolated:** New WebSocket connection handling code introduced

5.2.4 Root Cause Report & Remediation

PRIMARY ROOT CAUSE (91% confidence):
Code deployment v2.14.3 introduced memory leak in WebSocket handler
TECHNICAL DETAILS:
WebSocket connections not properly released from connection pool after client disconnect. Each orphaned connection retains 2MB of buffer memory. Over 48 hours, 1,800 leaked connections accumulated 3.6GB leaked memory.
CAUSAL CHAIN:
Code bug → Leaked WebSocket connections → Memory growth → GC pressure → CPU spikes (GC overhead) → Event processing delays → Queue backlog → Timeout cascade → Service failure
REMEDIATION:

- **Immediate:** Rollback to v2.14.2, restart all instances
- **Fix:** Add connection.close() in WebSocket disconnect handler
- **Prevention:** Add memory leak detection tests; implement connection pool monitoring

Case Study 3: Financial Services - Network Partition Causing Split-Brain

5.3.1 Scenario Background

System: High-frequency trading platform processing 100,000 transactions/second
Architecture: Multi-region deployment (US-East, US-West), Redis cluster (6 nodes), Kafka for event streaming
Incident: Data inconsistency causing duplicate trade executions
Business Impact: $2.3M in erroneous trades before detection

5.3.2 AI Detection: Multi-Metric Correlation

The system detected simultaneous anomalies across multiple dimensions:

Metric	Anomaly	Significance
cross_region_latency	45ms → 5000ms (timeout)	Network partition detected
redis_cluster_nodes	6 nodes → 2 clusters (3+3)	Split-brain condition
duplicate_transaction_rate	0.001% → 2.1%	Business logic failure

5.3.3 Causal Graph Analysis

The AI constructed the following causal graph:

Network Partition
↓ ↓
Redis Split (East) Redis Split (West)
↓ ↓
Both clusters accept writes
↓
Conflicting transaction state
↓
Duplicate trade executions

5.3.4 Root Cause & Resolution

ROOT CAUSE (95% confidence):
AWS inter-region network partition caused by upstream provider BGP route leak
SYSTEM VULNERABILITY:
Redis cluster lacked proper quorum configuration. Both partitions continued accepting writes, causing data divergence.
IMMEDIATE REMEDIATION:

- Halt all trading (circuit breaker)
- Manual reconciliation of transaction state

- Reverse erroneous trades ($2.3M)

LONG-TERM FIX:

- Implement Redis Sentinel with strict quorum (majority voting)
- Add cross-region connectivity health checks with automatic failover
- Deploy distributed consensus protocol (Raft/Paxos) for critical state

Case Study 4: IoT Platform - Thread Pool Exhaustion in Manufacturing System

5.4.1 Scenario Background

System: Industrial IoT platform monitoring 10,000 factory sensors
Architecture: Java Spring Boot API, MQTT broker, InfluxDB time-series database
Incident: API completely unresponsive; all requests timing out
Detection Time: 3 minutes (traditional monitoring would take 15-30 minutes)

5.4.2 AI Analysis: Rapid Root Cause Identification

Timeline:

- **15:42:00** - Sensor data ingestion job starts (scheduled daily task)
- **15:42:30** - API latency increases 150ms → 800ms
- **15:43:15** - Thread pool utilization 60/100 → 100/100
- **15:43:30** - All API requests timing out (complete failure)
- **15:45:00** - RCA system identifies root cause and recommends action

5.4.3 Causal Inference Results

Granger causality analysis revealed:

- **sensor_ingestion_job_active** → **influxdb_query_latency** ($p < 0.001$)
- **influxdb_query_latency** → **thread_pool_utilization** ($p < 0.001$)
- **thread_pool_utilization** → **api_timeout_rate** ($p < 0.001$)

5.4.4 AI-Generated Report

ROOT CAUSE (93% confidence):
Sensor data ingestion job blocking API thread pool
TECHNICAL EXPLANATION:
The daily sensor data ingestion job (processing 24 hours of historical data) executes synchronous InfluxDB queries on the shared API thread pool. Each query takes 8-12 seconds due to large data volume. With 100 concurrent queries, all 100 threads become blocked, preventing API from serving user requests.
IMMEDIATE FIX:

- Kill ingestion job process (immediate relief)
- Restart API service (restore normal operation)

PERMANENT SOLUTION:

- Move ingestion job to dedicated thread pool (isolation)
- Implement async processing with CompletableFuture
- Add rate limiting on background jobs (max 10 concurrent queries)

Case Study 5: Cloud Storage - DNS Propagation Delay Causing Regional Outage

5.5.1 Scenario Background

System: Cloud object storage service (similar to S3) serving 1PB of data
Architecture: Multi-region deployment with DNS-based load balancing
Incident: US-West region API endpoints returning 50% error rate
Complexity: Intermittent failure pattern; different users experiencing different behavior

5.5.2 AI Detection: Partial Failure Pattern

The LSTM model identified an unusual failure pattern:

- **Error rate by region:**

• US-East: 0.1% (normal)
• US-West: 47.3% (anomaly score: 0.97)
• EU-Central: 0.2% (normal)
• Asia-Pacific: 51.2% (anomaly score: 0.98)

- **Pattern analysis:** Specific IP ranges affected; others working fine within same region

5.5.3 Causal Analysis: Event Correlation

The system correlated the failure with a recent infrastructure change:

- **Event:** DNS record update 2 hours before failure (migrating to new load balancer IPs)
- **Failure pattern:** Clients with cached old DNS entries succeeding; clients with new DNS entries failing
- **Root cause:** New load balancer IPs not yet added to firewall whitelist

5.5.4 AI-Generated Diagnosis

ROOT CAUSE (89% confidence):
DNS record updated to new load balancer IPs, but firewall rules not updated
CAUSAL CHAIN:
DNS update → 50% clients resolve to new IPs → Firewall blocks new IPs → Connection refused errors → 50% failure rate
INSIGHT:
The AI system identified this as a “configuration drift” pattern where related configuration changes were not applied atomically. Traditional monitoring would flag “high error rate” but wouldn’t identify the DNS/firewall mismatch.
REMEDIATION:

- **Immediate:** Add new load balancer IPs to firewall whitelist (2-minute fix)
- **Process improvement:** Implement pre-deployment validation checks
- **Automation:** Create Terraform module linking DNS and firewall changes atomically

6. Technical Implementation Details

6.1 Technology Stack

Category	Technology	Purpose
Programming	Python 3.9+	Main development language
Deep Learning	PyTorch 2.0+	Neural network training and inference
Time Series Analysis	statsmodels, tsfresh	Statistical models, Granger causality
Causal Inference	DoWhy, CausalNex, tigramite	Causal graph construction, analysis
Graph Processing	NetworkX	Causal graph algorithms
Visualization	Plotly, Streamlit, D3.js	Interactive dashboards and graphs
Monitoring Integration	Prometheus, CloudWatch	Metrics collection from production
Deployment	Docker, Kubernetes	Containerization and orchestration

6.2 System Architecture Overview

The RCA system consists of six primary components operating in a sequential pipeline:

6. **Data Ingestion Layer:** Collects metrics from Prometheus/CloudWatch APIs, event logs from CI/CD systems, and dependency graphs from service mesh metadata.
7. **Preprocessing Pipeline:** Normalizes data, handles missing values, performs feature engineering, and creates sliding time windows for model input.
8. **Deep Learning Engine:** LSTM/Transformer models trained on healthy data to learn normal behavioral patterns and predict expected metric values.
9. **Anomaly Detection Module:** Compares actual vs. predicted values using reconstruction error and forecast deviation to identify anomalies with confidence scores.
10. **Causal Inference Engine:** Applies Granger causality tests, builds causal graphs using PC algorithm, and performs temporal precedence analysis.
11. **Report Generation & Visualization:** Ranks root causes, generates natural language explanations, creates interactive causal graphs, and integrates with incident management systems.

7. Expected Outcomes & Success Criteria

7.1 Performance Metrics & Targets

Metric Category	Metric	Target
Anomaly Detection Performance	Precision	**>80%**
	Recall	**>75%**
	F1 Score	**>0.77**
Root Cause Identification	Top-1 Accuracy	**>60%**
	Top-3 Accuracy	**>85%**
	Mean Reciprocal Rank	**>0.70**
System Performance	Detection Latency	**<5 minutes**
	Inference Time	**<10 seconds**
	Memory Footprint	**<4GB RAM**

7.2 Three Levels of Project Success

Pass Level - Minimum Viable Product

- Detects 5-10 anomaly types with >70% recall on synthetic data
- Identifies correct root cause in top-3 for >60% of failures
- Working end-to-end pipeline from data ingestion to report generation
- Basic web dashboard showing anomaly scores and top root causes
- Comprehensive documentation (30+ pages)

Distinction Level - Production-Quality System

- Detects 20+ anomaly types with >80% precision and recall
- Top-1 root cause accuracy >65% on public benchmarks
- Integration with real monitoring system (Prometheus or CloudWatch)
- Interactive causal graph visualization with drill-down capabilities
- Evaluation on multiple public datasets (NASA, Azure)
- Docker containerization with deployment instructions
- Clean, modular, well-documented code on GitHub

Publication-Worthy - Research Contribution

- Novel causal inference technique or neural architecture

- Statistically significant improvement over state-of-the-art baselines
- Deployment in real production environment with measured business impact
- Quantified MTTR reduction (e.g., 4 hours → 15 minutes)
- Open-source release with documentation and tutorials
- Conference/workshop paper submission (AIOps, SREcon, ICSE)

8. Conclusion

The Automated Root Cause Analysis system represents a paradigm shift in how organizations diagnose production failures. By combining deep learning’s pattern recognition capabilities with causal inference’s ability to distinguish cause from effect, this project addresses a critical gap in current monitoring and observability solutions.

8.1 Why This Project Stands Out

- **Real-World Impact:** Every organization running production systems struggles with root cause analysis. Google, Netflix, and Amazon have invested millions in solving this problem. This project provides a solution accessible to any engineering team.
- **Technical Depth:** Integrates machine learning (LSTM/Transformers), statistics (Granger causality, causal graphs), systems engineering (monitoring integration), and software engineering (production-grade code). Few student projects achieve this level of multi-disciplinary integration.
- **Measurable Success:** Unlike many AI projects with subjective outcomes, this system has clear, quantifiable metrics: precision, recall, top-k accuracy, detection latency. Success or failure is objectively measurable.
- **Career Relevance:** Directly demonstrates skills for Site Reliability Engineer, DevOps Engineer, Platform Engineer, and ML Engineer roles. Companies actively seek engineers who understand both ML and production systems.
- **Research Potential:** Novel contributions in causal inference architectures, temporal modeling techniques, or real-world deployment insights could lead to academic publications.

8.2 Key Takeaways from Case Studies

The five case studies demonstrate the system’s versatility:

- **Case 1 (E-commerce):** Identified delayed effect (7 hours) from database migration to user-facing failure
- **Case 2 (SaaS):** Detected gradual memory leak pattern invisible to traditional monitoring
- **Case 3 (Finance):** Diagnosed complex distributed system failure (network partition → split-brain)
- **Case 4 (IoT):** Rapid root cause identification (3 minutes vs. traditional 15-30 minutes)
- **Case 5 (Cloud Storage):** Identified configuration drift pattern (DNS/firewall mismatch)

These diverse scenarios prove the system’s applicability across industries and failure types.

8.3 Next Steps for Implementation

12. **Immediate (Week 1):** Set up development environment, review key research papers, create project plan
13. **Short-term (Weeks 2-4):** Build synthetic data generator, implement baseline LSTM model, achieve first anomaly detection
14. **Medium-term (Weeks 5-8):** Add causal inference layer, implement root cause ranking, test on synthetic failures
15. **Long-term (Weeks 9-16):** Integrate with real monitoring, build visualization dashboard, comprehensive evaluation, final documentation

8.4 Final Thoughts

This project bridges the gap between academic machine learning and practical systems engineering. It creates a tool that could genuinely improve how organizations handle production incidents, potentially saving millions of dollars in downtime costs and hundreds of engineering hours in manual investigation.

More importantly, it demonstrates a holistic understanding of modern software systems: how they fail, how to instrument them, how to apply AI to real-world problems, and how to build production-quality solutions. These skills are increasingly valuable as systems grow more complex and organizations seek engineers who can work at the intersection of ML and infrastructure.

With proper execution, this work has genuine potential for publication, industry adoption, and significant career impact.

— End of Document —

© 2026 | For Educational and Project Use